

Centres étrangers – 2026 – sujet1 - Correction

Exercice 1 (6 points)

Partie A : Dictionnaire et tri

1. Calcul du score de l'athlète Alex

- 100 m : $(20 - 13,0) \times 10 = 70$
- Saut en longueur : $5,2 \times 20 = 104$
- Lancer de poids : $9,0 \times 10 = 90$
- 1500 m : $500 - 310 = 190$

Score total : $70 + 104 + 90 + 190 = 454$

Réponse : Alex obtient **454 points**.

👉 *Commentaire :* La question consiste simplement à appliquer les formules fournies puis à additionner les résultats obtenus pour chaque épreuve.

2. Compléter la fonction nb_points

```
def nb_points(epreuve, valeur):  
    points = 0  
    if epreuve == '100m':  
        points = (20 - valeur) * 10  
    elif epreuve == 'longueur':  
        points = valeur * 20  
    elif epreuve == 'poids':  
        points = valeur * 10  
    elif epreuve == '1500m':  
        points = 500 - valeur  
    return points
```

👉 *Commentaire :* Les formules sont directement celles données dans l'énoncé. La valeur retournée est stockée dans la variable `points`.

3. Compléter la fonction `score`

```
def score(athlete):  
    total = 0  
    performances = athlete['performances']  
    for epreuve in performances:  
        valeur = performances[epreuve]  
        total += nb_points(epreuve, valeur)  
    athlete['score'] = total
```

👉 *Commentaire : On parcourt toutes les épreuves du dictionnaire puis on ajoute les points correspondants au score total.*

4. Compléter la fonction `classer`

Ligne 6 : `if l[j]['score'] > l[max_index]['score']:`

👉 *Commentaire : On recherche à chaque étape l'athlète possédant le score maximal dans la partie non encore triée de la liste.*

5. Identifier le tri utilisé

Réponse : tri par sélection.

👉 *Commentaire : À chaque passage, l'algorithme sélectionne le plus grand élément puis l'échange avec la première position disponible.*

6. Coût de la fonction `classer`

Nombre de comparaisons : $(n - 1) + (n - 2) + \dots + 2 + 1 = \frac{n(n-1)}{2}$

Complexité : $O(n^2)$

👉 *Commentaire : Le nombre de comparaisons correspond à celui du tri par sélection, qui est quadratique.*

Partie B : Arbre binaire de recherche

7. Créer l'objet alex

```
alex = Athlete("Alex", 13.0, 5.2, 9.0, 310.0)
```

👉 *Commentaire : Les paramètres doivent être fournis dans l'ordre défini par le constructeur de la classe.*

8. Écrire la méthode calculer_score

```
def calculer_score(self):  
    return ((20 - self.m100) * 10  
            + self.longueur * 20  
            + self.poids * 10  
            + (500 - self.m1500))
```

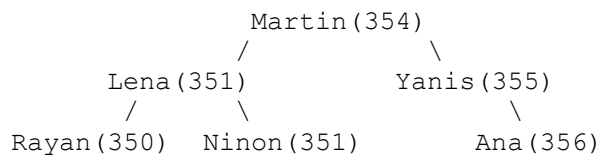
👉 *Commentaire : Le score est obtenu en appliquant les mêmes formules que dans la partie A, mais cette fois à partir des attributs de l'objet.*

9. Construire l'arbre binaire de recherche

Insertion dans l'ordre :

- Martin (354)
- Lena (351)
- Rayan (350)
- Yanis (355)
- Ninon (351)
- Ana (356)

Arbre obtenu :



👉 *Commentaire : Les scores strictement inférieurs sont placés à gauche ; les scores supérieurs ou égaux sont placés à droite.*

10. Compléter la méthode `insérer`

```
def insérer(self, athlete):
    if athlete.score < self.valeur.score:
        if self.gauche == None:
            self.gauche = Noeud(athlete)
        else:
            self.gauche.insérer(athlete)
    else:
        if self.droite == None:
            self.droite = Noeud(athlete)
        else:
            self.droite.insérer(athlete)
```

👉 *Commentaire : L'insertion est récursive : on descend dans le sous-arbre gauche ou droit jusqu'à trouver une place libre.*

11. Expliquer le parcours réalisé par `classer`

Réponse :

La méthode effectue un **parcours infixe inversé** :

1. parcours du sous-arbre droit ;
2. visite du nœud courant ;
3. parcours du sous-arbre gauche.

Comme les plus grands scores se trouvent dans le sous-arbre droit, les athlètes sont visités du score le plus élevé vers le plus faible.

👉 *Commentaire : Un parcours infixe classique donnerait les scores dans l'ordre croissant. En inversant gauche et droite, on obtient l'ordre décroissant.*

12. Comparer les deux approches

a. Dictionnaires et tri

Avantage :

- Solution simple à programmer et à comprendre.

Inconvénient :

- Le tri coûte $O(n^2)$ avec l'algorithme proposé.

👉 *Commentaire : Cette approche est adaptée à de petites listes mais devient moins efficace lorsque le nombre d'athlètes augmente.*

b. Arbre binaire de recherche orienté objet

Avantage :

- L'insertion et la recherche peuvent être rapides, en moyenne $O(\log n)$

Inconvénient :

- Mise en œuvre plus complexe et performances dégradées si l'arbre devient déséquilibré.

👉 *Commentaire : Cette approche est intéressante lorsque les données sont ajoutées progressivement et qu'il faut maintenir un classement dynamique.*
